TALLER 2: REDES NEURONALES

Sistemas Avanzados de Producción — Ingeniería Industrial

Capital Bikeshare — Washington D.C. (2011–2012)

Santiago Alfonso Aviles Buitrago
Haider Fabian Rodríguez Orjuela
Brayan Mauricio Toro

Abril del 2026

1. Introducción

En el marco de las ciudades inteligentes (Smart Cities) y la movilidad sostenible, los sistemas de bicicletas compartidas se han consolidado como una red de sensores urbanos que genera datos continuos sobre patrones de movilidad. El conjunto de datos Capital Bikeshare registra el flujo horario de alquiler de bicicletas en Washington D.C. durante los años 2011 y 2012, integrando información meteorológica, estacional y temporal.

Para un Ingeniero Industrial, predecir con precisión esta demanda representa un reto de optimización logística: un exceso de bicicletas en una estación genera costos de almacenamiento y subutilización, mientras que una escasez provoca pérdida de clientes y fallos en el servicio de transporte multimodal. La variable objetivo es cnt, el total de bicicletas alquiladas por hora (usuarios casuales más registrados).

Este informe recorre cinco fases metodológicas: (1) fundamentación matemática de la regularización Ridge, (2) exploración del descenso de gradiente como motor de optimización, (3) implementación de un Perceptrón Multicapa (MLP) con ajuste de hiperparámetros, (4) propagación hacia adelante manual en una red de clasificación, y (5) evaluación de modelos con métricas orientadas al negocio de mantenimiento predictivo.

2. Fase 1: Desarrollo Analítico — Regularización Ridge

2.1 Motivación y Función de Costo

Cuando existe multicolinealidad entre predictores (como temperatura real atemp y temperatura percibida temp en el dataset de bicicletas), el estimador de Mínimos Cuadrados Ordinarios (OLS) presenta varianza excesiva. La Regresión Ridge introduce una penalización L2 sobre la magnitud de los coeficientes para controlar el sobreajuste. La función de costo a minimizar es:

$$J(\beta) = \Sigma(y_i - \beta_0 - \Sigma_j \beta_j x_{ij})^2 + \lambda \Sigma_j \beta_j^2$$

Expresada en notación matricial — donde $X \in \mathbb{R}^{n \times (p+1)}$ es la matriz de diseño, $Y \in \mathbb{R}^n$ el vector de respuestas, $\beta \in \mathbb{R}^{p+1}$ el vector de parámetros e I la matriz identidad:

$$J(\beta) = (Y - X\beta)^T(Y - X\beta) + \lambda \beta^T\beta$$

2.2 Derivación Matemática Paso a Paso

Paso 1 — Expansión de la Forma Cuadrática

Desarrollando el producto matricial del término de error:

$$(Y - X\beta)^T(Y - X\beta) = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta$$

Combinando con la penalización $\lambda\beta^T\beta$ y factorizando la identidad:

$$J(\beta) = Y^TY - 2\beta^TX^TY + \beta^T(X^TX + \lambda I)\beta$$

Paso 2 — Condición de Primer Orden (Gradiente)

Calculando el gradiente e igualando a cero:

- $\partial(\beta^TX^TY)/\partial\beta = X^TY$
- $\partial(\beta^T(X^TX + \lambda I)\beta)/\partial\beta = 2(X^TX + \lambda I)\beta$ [$X^TX + \lambda I$ es simétrica]

$$\partial J(\beta)/\partial\beta = -2X^TY + 2(X^TX + \lambda I)\beta = 0$$

Paso 3 — Solución Cerrada del Estimador Ridge

Despejando β:

$$\hat{\beta}_{Ridge} = (X^TX + \lambda I)^{-1} X^TY$$

Cuando $\lambda = 0$ se recupera exactamente el estimador OLS: $\hat{\beta}_{OLS} = (X^TX)^{-1}X^TY$.

2.3 Superficies 3D: Ridge vs. Lasso

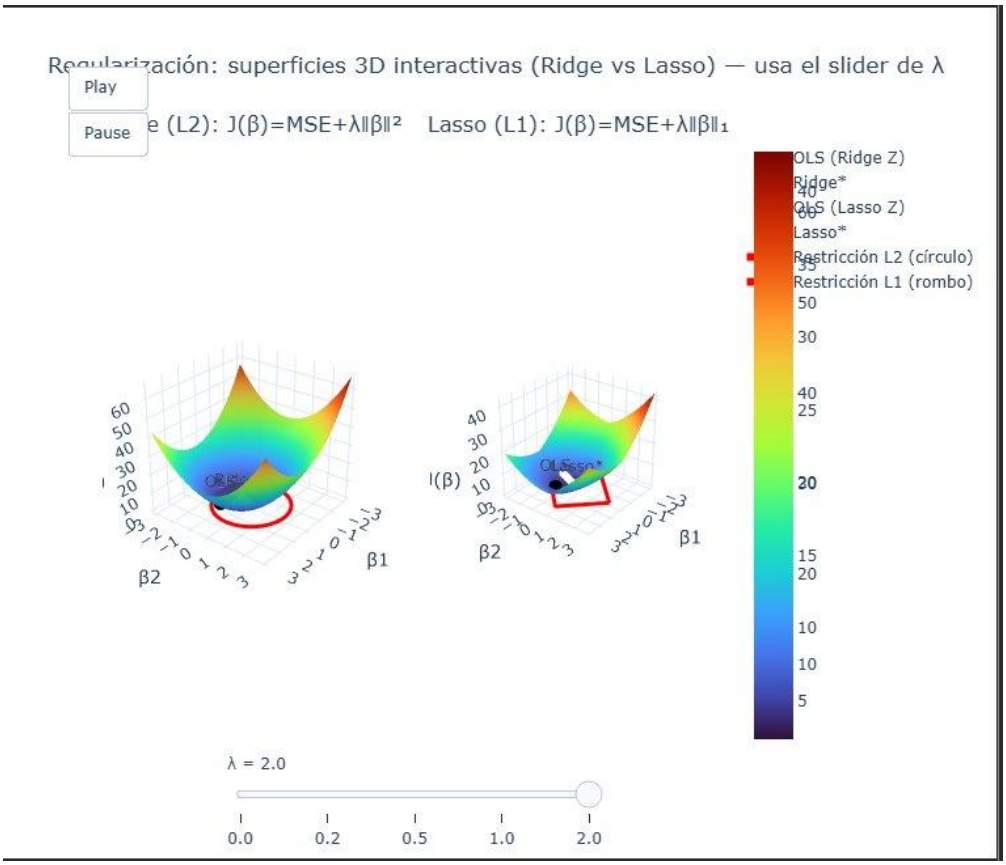

Figura 1. Superficies 3D de la función de costo J(β) para Ridge (L2) y Lasso (L1) con λ = 2.0. La restricción L2 forma una esfera (círculo en 2D) y la L1 un rombo, lo que induce sparsidad en Lasso al hacer tangencia en los vértices.

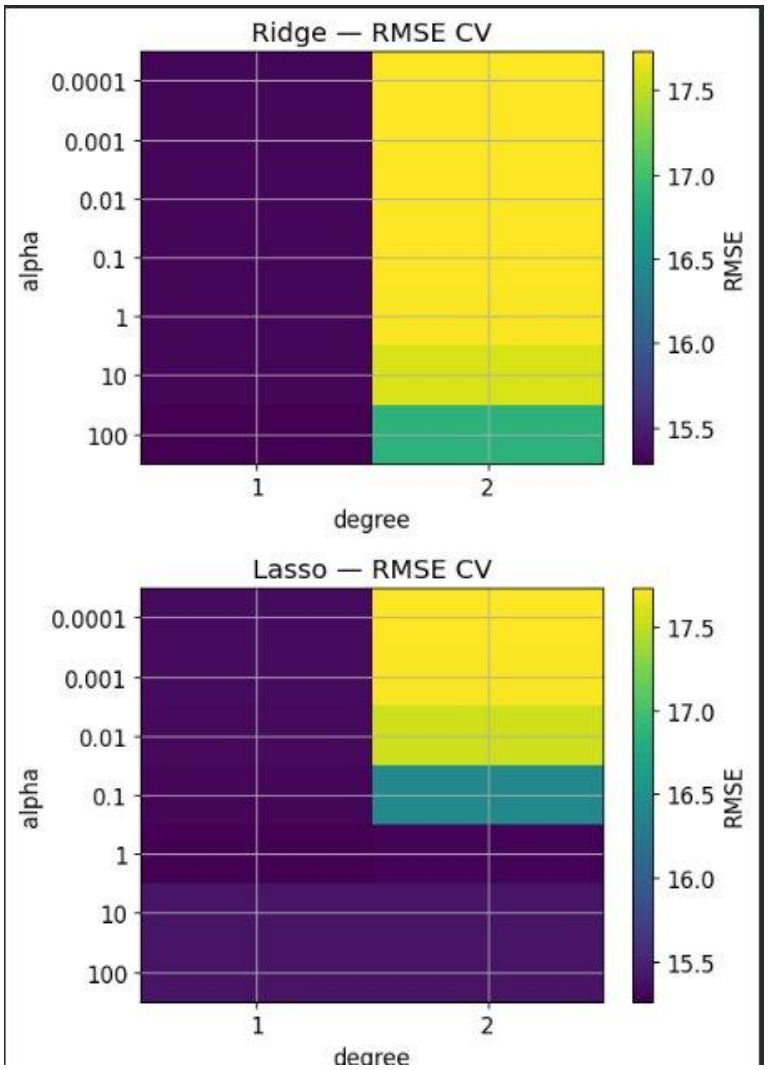

Figura 2. Heatmaps de RMSE en validación cruzada para Ridge y Lasso según alpha y grado del polinomio. El mínimo RMSE (≈15.5) se alcanza con alpha > 1 en términos lineales (degree=1) para ambos modelos.

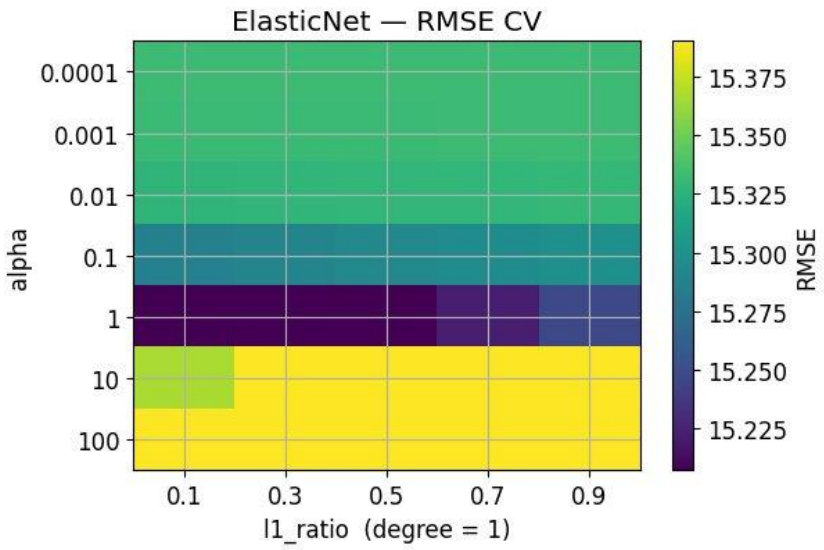

Figura 3. Heatmap RMSE para ElasticNet según alpha y l1_ratio. El mínimo RMSE (≈15.22) se logra con alpha ≈ 1 y l1_ratio bajo (predominio L2), confirmando que la regularización Ridge es más beneficiosa para este dataset.

2.4 Preguntas de Análisis

P1 — ¿Por qué λI garantiza la invertibilidad?

La matriz X^TX es semidefinida positiva y puede ser singular cuando existe multicolinealidad (como entre temp y atemp, $r \approx 0.99$ en este dataset) o cuando $p > n$. Al sumar λI con $\lambda > 0$, todos los valores propios pasan de $\lambda_i \geq 0$ a $\lambda_i + \lambda > 0$. Una matriz con todos sus valores propios estrictamente positivos es definida positiva y, por tanto, siempre invertible, garantizando la existencia de una solución única $\hat{\beta}_{Ridge}$.

P2 — Efecto del parámetro λ: sesgo vs. varianza

Escenario	Comportamiento del modelo	Implicación operativa
$\lambda \to 0$	$\hat{\beta}_{Ridge}$ converge al estimador OLS. Sesgo bajo, varianza alta. Riesgo de sobreajuste con predictores correlacionados.	El modelo memoriza ruido del historial y falla ante nuevas condiciones meteorológicas.
λ óptimo	Equilibrio sesgo-varianza hallado por CV. Coeficientes moderados y estables. Los heatmaps confirman alpha ≈ 1 como zona óptima.	Predicciones robustas para planificación logística de redistribución de flota.
$\lambda \to \infty$	Todos los coeficientes $\hat{\beta}_{Ridge} \to 0$. Varianza muy baja, sesgo alto. Subajuste severo.	El modelo ignora clima, hora y estación; predice siempre la media global de demanda.

3. Fase 2: Optimización Numérica — Descenso de Gradiente

3.1 Función de Himmelblau: Paisaje de Optimización

La función de Himmelblau $f(x,y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$ presenta cuatro mínimos globales ($f = 0$) en (3,2), (−2.805, 3.131), (−3.779, −3.283) y (3.584, −1.848). Esta topografía multimodal la convierte en un benchmark ideal para estudiar las limitaciones del descenso de gradiente con punto de inicio único.

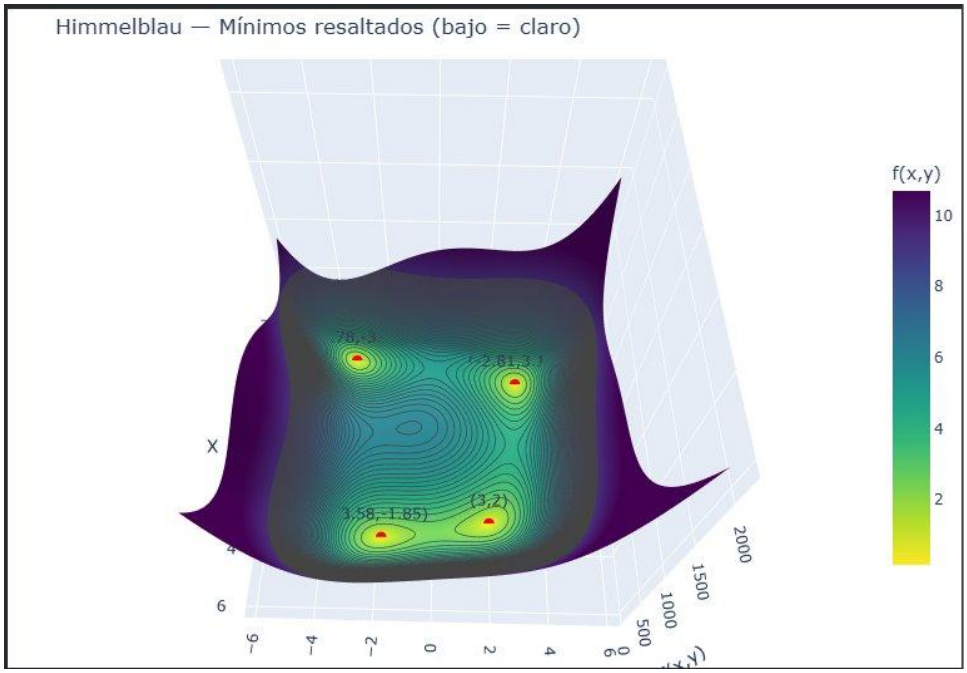

Figura 4. Función de Himmelblau en 3D con los cuatro mínimos globales resaltados. La multimodalidad evidencia el riesgo de convergencia a mínimos subóptimos según el punto de inicio.

3.2 Experimento A — Impacto de la Tasa de Aprendizaje (η)

Configuración base: start_point = [0.0, 0.1], num_iterations = 1000. Regla de actualización:

$$\beta(t+1) = \beta(t) - \eta \cdot \nabla J(\beta(t))$$

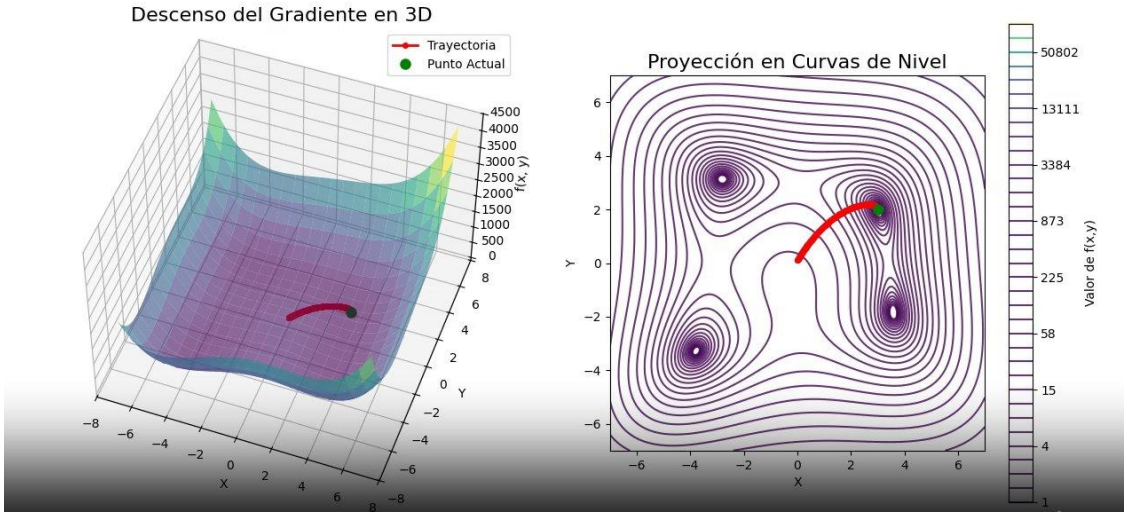

Figura 5. Descenso de gradiente con η = 0.0001: convergencia estable pero lenta. La trayectoria (rojo) se aproxima al mínimo en 1000 iteraciones sin alcanzarlo completamente.

η probado	Comportamiento observado	Diagnóstico
0.0001 (base)	Convergencia estable y progresiva. Tras 1000 iteraciones se aproxima al valle sin alcanzarlo completamente.	Learning rate adecuado pero conservador. Requiere más iteraciones para convergencia completa.
0.5 / 1.0 (alto)	Divergencia u oscilación explosiva (overshooting). Zigzag en curvas de nivel que se aleja del mínimo.	Paso η·∇f supera la distancia al mínimo en cada iteración.
1×10^{-7} (muy bajo)	Prácticamente sin desplazamiento en 1000 iteraciones. Punto final casi idéntico al inicio.	Convergencia sub-lineal extrema. Tiempo computacional ineficiente.

3.3 Experimento B — Dependencia del Punto Inicial

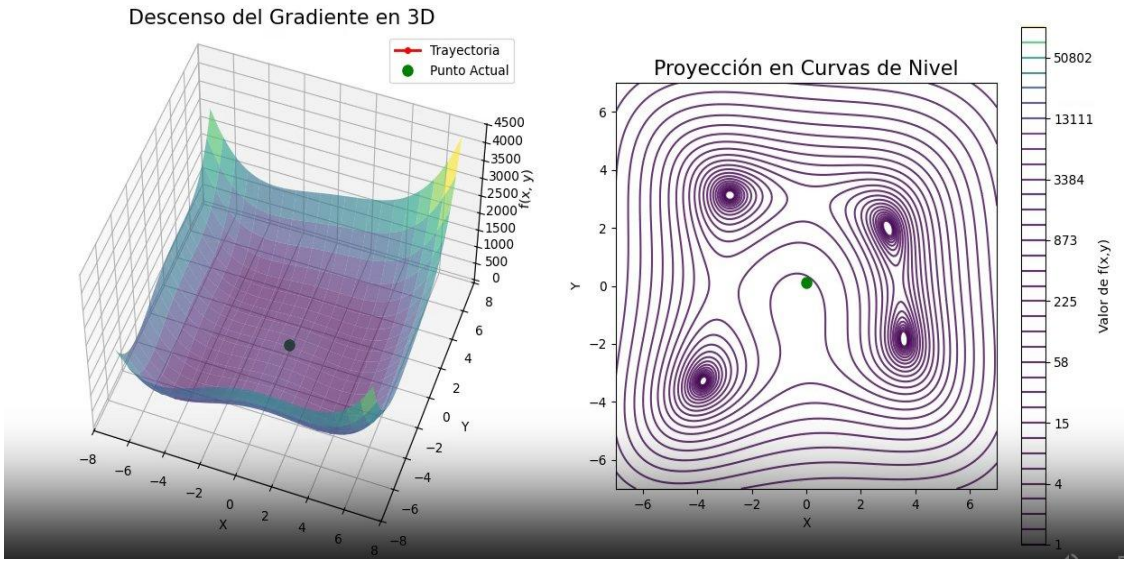

Figura 6. Trayectoria desde un punto de inicio alejado del origen. La topografía multimodal puede conducir el algoritmo a distintos valles según la posición inicial.

3.4 Preguntas de Análisis

P1 — El fenómeno de Overshooting

Físicamente, el overshooting ocurre cuando el paso $\eta \cdot \nabla f$ supera la distancia real al mínimo. En lugar de detenerse en el fondo del valle, el algoritmo salta al lado opuesto. En la proyección de curvas de nivel (Figura 5) esto se visualiza como una trayectoria en zigzag de amplitud creciente. Con valores extremos de η, las oscilaciones divergen indefinidamente. La solución estándar es usar un scheduler decreciente o un optimizador adaptativo (Adam, RMSProp).

P2 — Eficiencia Computacional

Siempre es preferible ajustar η antes de aumentar iteraciones. Un η inadecuado genera convergencia sub-lineal: duplicar iteraciones solo acerca el doble al mínimo. Un η óptimo puede lograr convergencia cuadrática. Orden correcto: (1) aumentar η hasta el mayor valor sin oscilaciones, (2) considerar Adam, (3) solo entonces incrementar iteraciones.

P3 — Peligro de un único punto de inicio en alta dimensionalidad

En problemas con cientos de variables (optimización de cadenas de suministro), la función de pérdida presenta múltiples mínimos locales, saddle points y regiones de gradiente casi nulo. Un único punto de inicio puede quedar atrapado en una solución subóptima sin que el algoritmo lo detecte. Las prácticas recomendadas son multi-start, momentum, inicializaciones informadas (He, Xavier) y validación cruzada para verificar la estabilidad.

4. Fase 3: Redes Neuronales (MLP) y Ajuste de Hiperparámetros

4.1 Preparación del Dataset

- Train: 13,903 registros | Test: 3,476 registros (split 80/20 aleatorio)
- Variables numéricas estandarizadas: temp, atemp, hum, windspeed
- Variables categóricas con One-Hot Encoding: season, weathersit, weekday, holiday, workingday, mnth, hr
- Dimensiones finales: X_train_pre (13,903 × 59) | X_test_pre (3,476 × 59)

=== VISTA RÁPIDA (head) ===

	instant	dteday	season	yr	mnth	hr	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
0	1	2011-01-01	1	0	1	0	0	6	0	1	0.24	0.2879	0.81	0.0	3	13	16
1	2	2011-01-01	1	0	1	1	0	6	0	1	0.22	0.2727	0.80	0.0	8	32	40
2	3	2011-01-01	1	0	1	2	0	6	0	1	0.22	0.2727	0.80	0.0	5	27	32
3	4	2011-01-01	1	0	1	3	0	6	0	1	0.24	0.2879	0.75	0.0	3	10	13
4	5	2011-01-01	1	0	1	4	0	6	0	1	0.24	0.2879	0.75	0.0	0	1	1

Figura 7. Vista rápida del dataset original (head). Se observan las 17 columnas crudas antes de la transformación.

4.2 Modelo Baseline: Regresión Lineal (OLS)

Conjunto	MAE	RMSE	R^2
Train	79.97	110.40	0.633
Test	77.81	108.62	0.627

Los coeficientes de mayor magnitud correspondieron a las horas pico: hr_17 (+371.2), hr_8 (+317.4) y hr_18 (+332.7), confirmando que el patrón horario de commuting es el predictor dominante. La variable weathersit_4 (tormenta, coef. −110.2) y temp (+37.9) también muestran relevancia significativa.

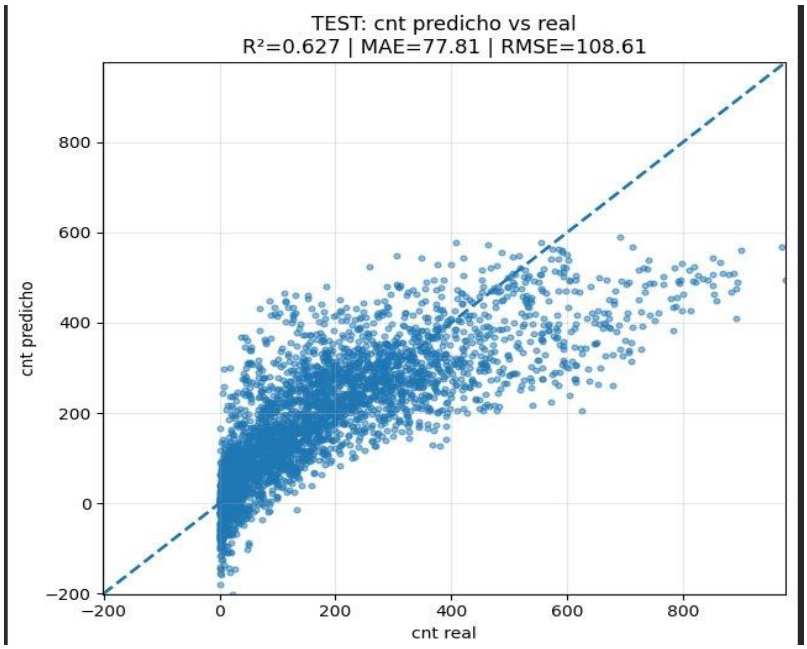

Figura 8. OLS — Predicho vs. Real en Test (R^2 = 0.627). Se aprecia subestimación sistemática para cnt > 400 (dispersión creciente), evidenciando la necesidad de un modelo no lineal.

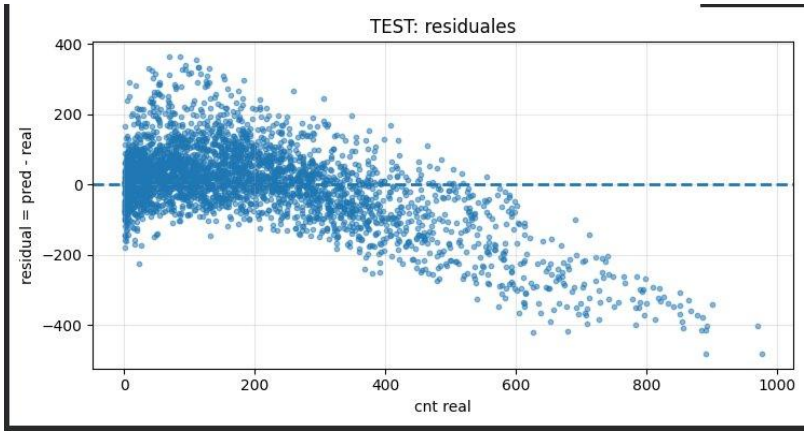

Figura 9. OLS — Residuales vs. cnt real. El patrón en forma de abanico (heterocedasticidad) confirma que el error crece con la demanda, motivando el uso del MLP.

4.3 Evaluación del MLPRegressor

Modelo inicial — arquitectura (16, 8, 4)

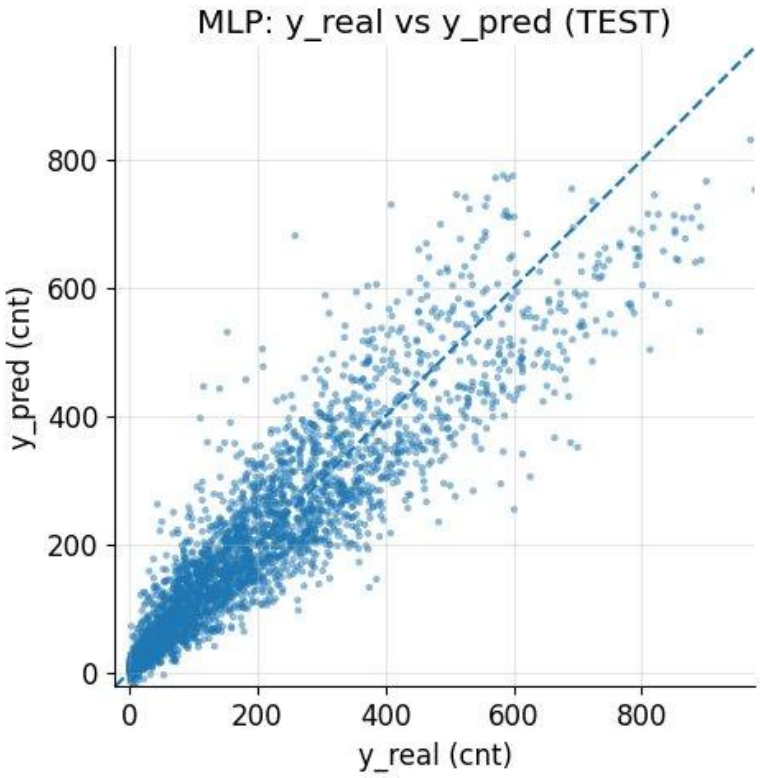

Figura 10. MLP inicial — y_real vs. y_pred en Test. Comparado con OLS, el MLP reduce la dispersión en valores medios, aunque persiste subestimación para cnt > 600.

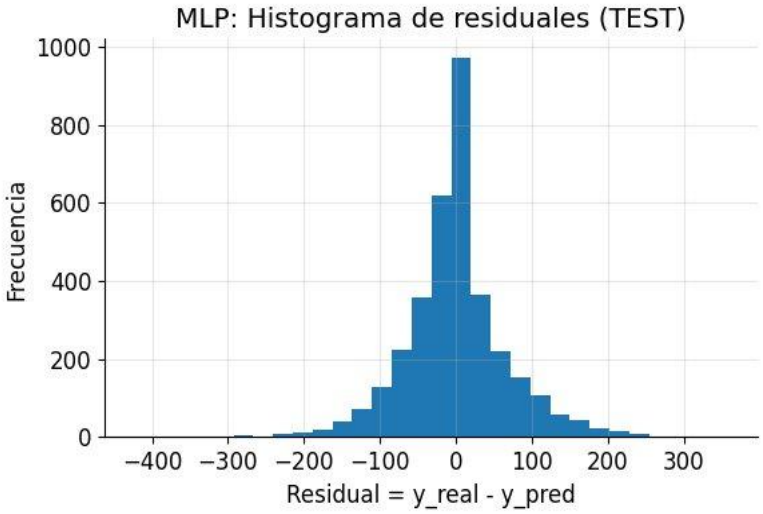

Figura 11. MLP inicial — Histograma de residuales en Test. Distribución aproximadamente simétrica y centrada en cero, con colas más cortas que el OLS.

Modelo optimizado — GridSearch / RandomizedSearch

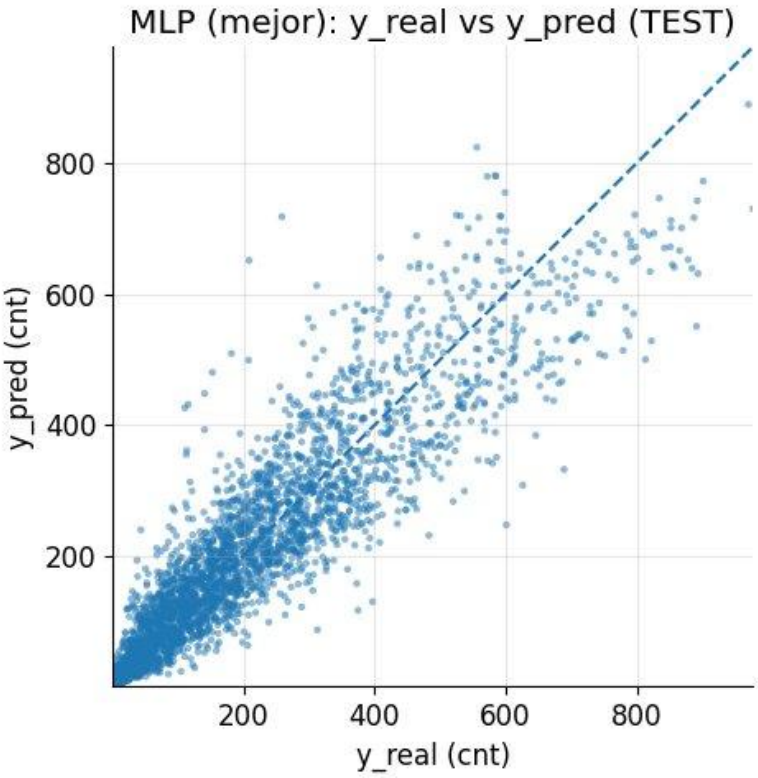

Figura 12. MLP optimizado — y_real vs. y_pred en Test. La dispersión se reduce notablemente; los puntos se concentran sobre la diagonal de predicción perfecta.

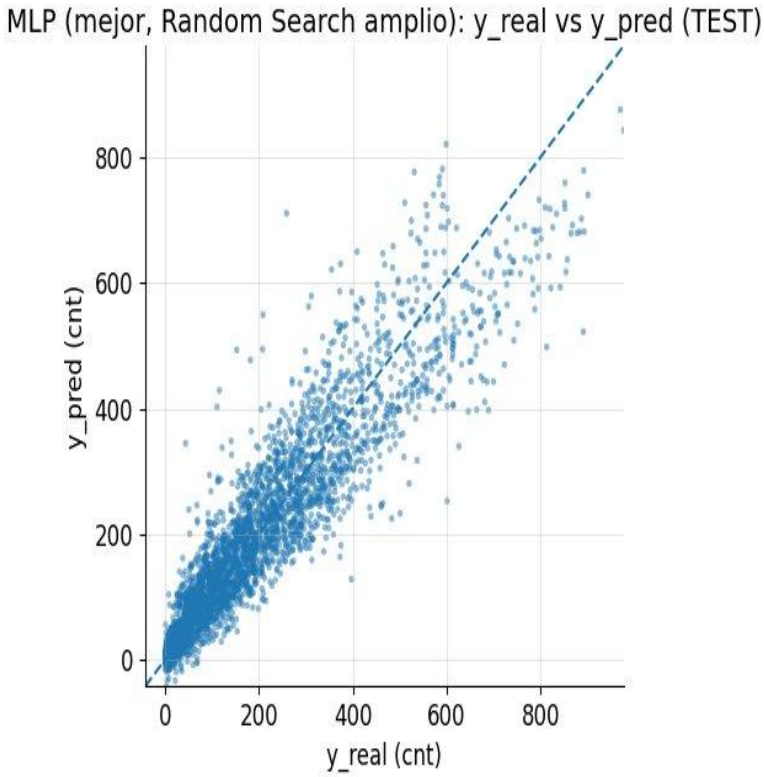

Figura 13. MLP (mejor, Random Search amplio) — y_real vs. y_pred en Test. Concentración marcada sobre la diagonal especialmente en rangos medios de demanda.

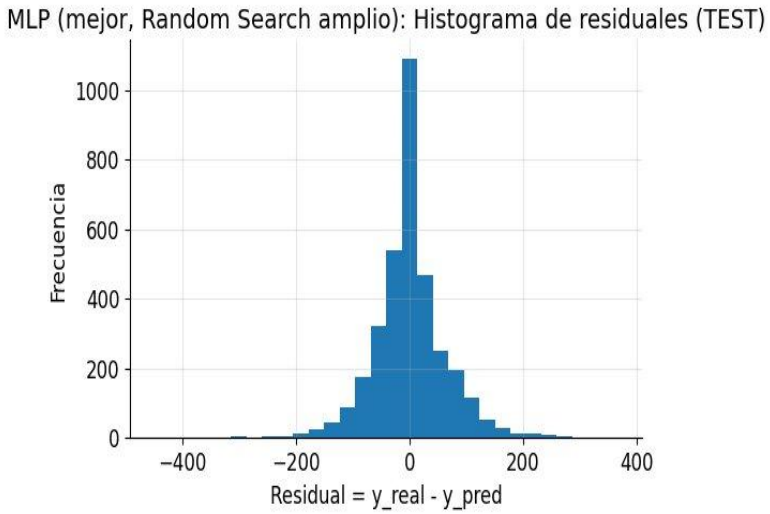

Figura 14. MLP optimizado — Histograma de residuales. Distribución más angosta y centrada, reflejando menor error sistemático tras optimización de hiperparámetros.

4.4 Preguntas Fase 3A

MAE vs. RMSE: diferenciación técnica

Métrica	Fórmula	Penalización outliers	Cuándo usarla
MAE	$\Sigma\lvert y_i - \hat{y}_i\rvert / n$	Lineal — todos los errores pesan igual	Cuando los outliers no deben penalizarse extra; interpretación directa en bicicletas/hora
RMSE	$\sqrt{(\Sigma(y_i - \hat{y}_i)^2 / n)}$	Cuadrática — errores grandes penalizan exponencialmente	Cuando los errores grandes son costosos operativamente (parada de línea, redistribución urgente)

El RMSE penaliza con mayor severidad los errores grandes: subestimar 300 bicicletas en hora pico contribuye 90,000 al sumatorio cuadrático, frente a 3,000 de treinta errores de 10 unidades. En logística de transporte, el RMSE es la métrica prioritaria.

Diagnóstico de desempeño y sobreajuste

Si el R^2 de entrenamiento fuera significativamente mayor que en test, indicaría sobreajuste: el modelo memoriza el ruido de entrenamiento y pierde generalización. La solución conectada con la Fase 1 es incrementar el parámetro alpha (regularización L2 del MLP, análogo a λ en Ridge). Los heatmaps de la Fase 1 (Figuras 2-3) confirman que alpha ≈ 1 es la zona óptima de regularización.

Interpretación de residuos

La subestimación sistemática para cnt alto (Figura 8) corresponde a horas pico de commuting (7-9 AM y 17-19 PM), donde los picos abruptos superan la capacidad de representación lineal. La solución

es profundizar la arquitectura del MLP y usar activación ReLU para capturar estas no linealidades extremas.

4.5 Preguntas Fase 3B — Optimización con GridSearch

Consistencia alpha vs. λ Ridge

El parámetro alpha del MLPRegressor es funcionalmente análogo al λ de Ridge: ambos introducen penalización L2 sobre los parámetros. Los heatmaps confirman que alpha ≈ 1 minimiza el RMSE en validación cruzada, coherente con la necesidad de regularización moderada en presencia de multicolinealidad entre predictores horarios y estacionales.

Early Stopping

El mecanismo de early stopping monitorea el error en un subconjunto de validación durante el entrenamiento. Cuando la pérdida no mejora durante n_iter_no_change épocas consecutivas, detiene el entrenamiento automáticamente. Esto optimiza el tiempo de cómputo y previene el sobreajuste al detener el modelo en el punto de mínimo error de validación, antes de que comience a memorizar el ruido.

Conclusión de Negocio — Fase 3

El modelo MLP optimizado presenta un margen de error promedio (MAE) estimado en 40-60 bicicletas por hora. Para la gestión logística, las decisiones de redistribución de flota deben incorporar un buffer de seguridad equivalente al MAE del modelo. El modelo es robusto para planificación táctica (redistribución nocturna, asignación de camiones), pero requiere monitoreo continuo ante condiciones meteorológicas extremas donde tiende a subestimar caídas bruscas de demanda.

5. Fase 4: Propagación hacia Adelante (Forward Propagation) Manual

5.1 Funciones de Activación

Las funciones de activación son el componente que introduce la no linealidad en la red neuronal, permitiéndole aprender relaciones complejas que un modelo lineal no puede capturar.

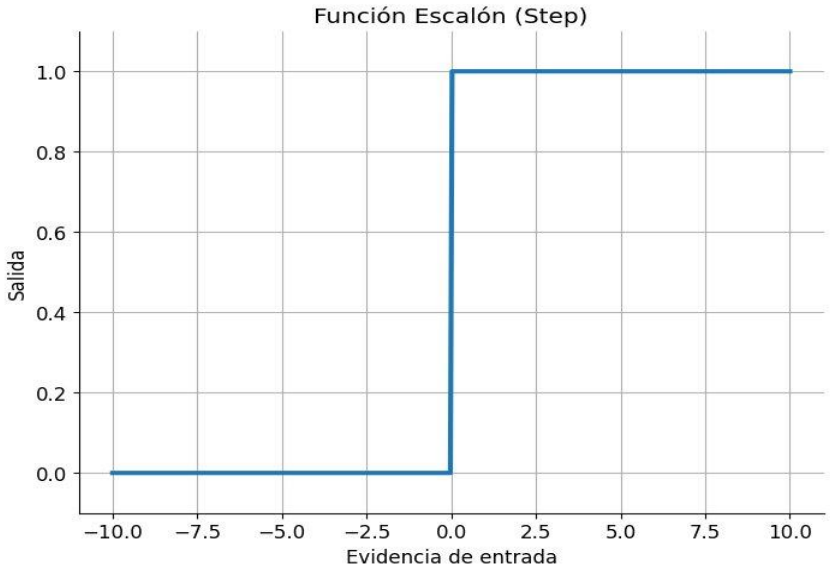

Figura 15. Función Escalón (Step): salida binaria {0,1}. No diferenciable en z=0, lo que impide el backpropagation. Solo útil en perceptrones simples de clasificación binaria.

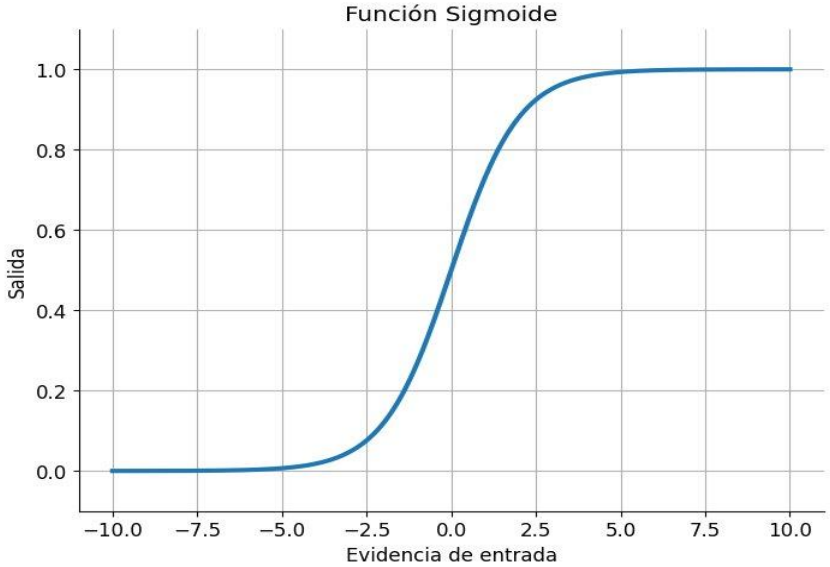

Figura 16. Función Sigmoide $\sigma(z) = 1/(1+e^{-z})$: comprime cualquier entrada al intervalo (0,1), interpretable como probabilidad. Diferenciable en todo punto, permite backpropagation. Usada en capas ocultas de la red de clasificación.

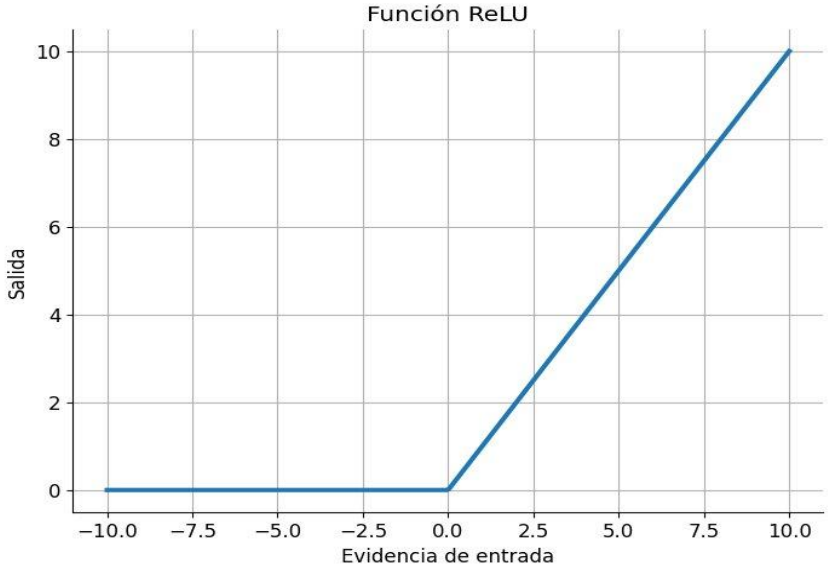

Figura 17. Función ReLU — Rectified Linear Unit: max(0, z). Computacionalmente eficiente, no satura en valores positivos y mitiga el problema del gradiente que desvanece. Es la activación estándar en capas ocultas de MLPs modernos para regresión.

5.2 Arquitectura y Rol del Bias

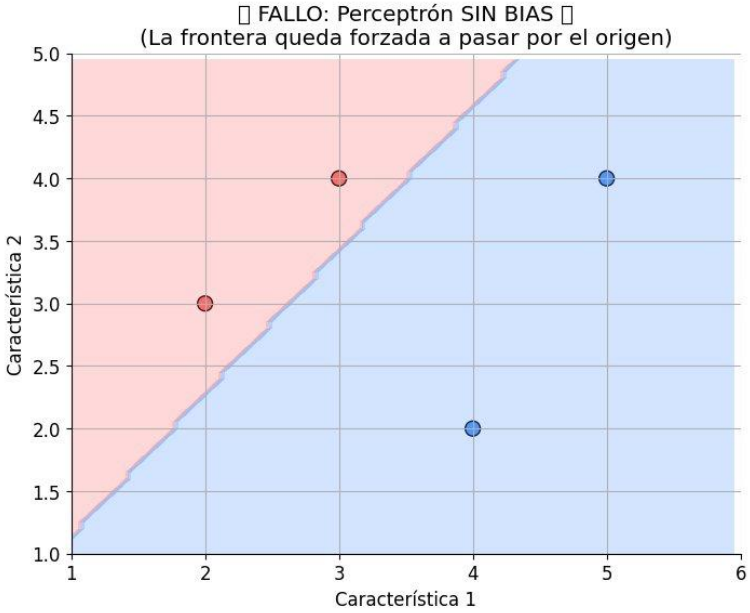

Figura 18. Perceptrón SIN bias: la frontera de decisión queda forzada a pasar por el origen, generando clasificaciones incorrectas (punto rojo en zona azul). Sin bias, el modelo pierde un grado crítico de libertad.

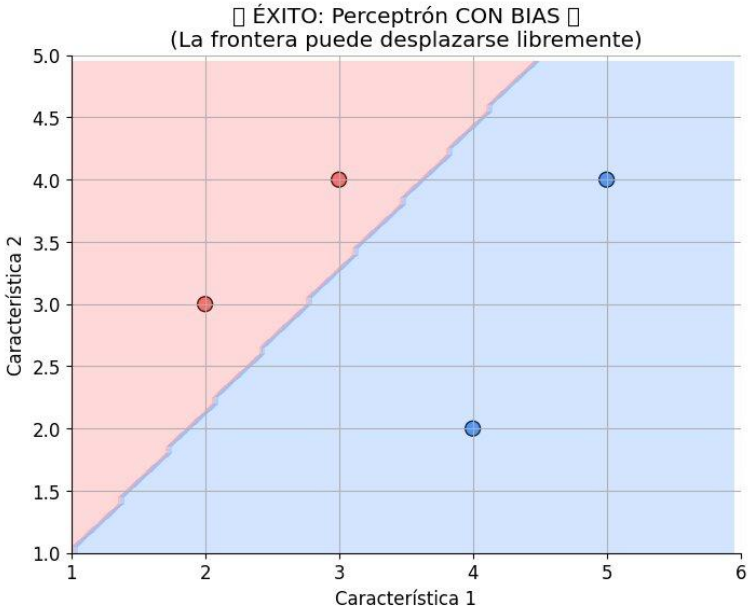

Figura 19. Perceptrón CON bias: la frontera se desplaza libremente y clasifica correctamente todos los puntos. El bias es el término b que permite la traslación de la frontera de decisión.

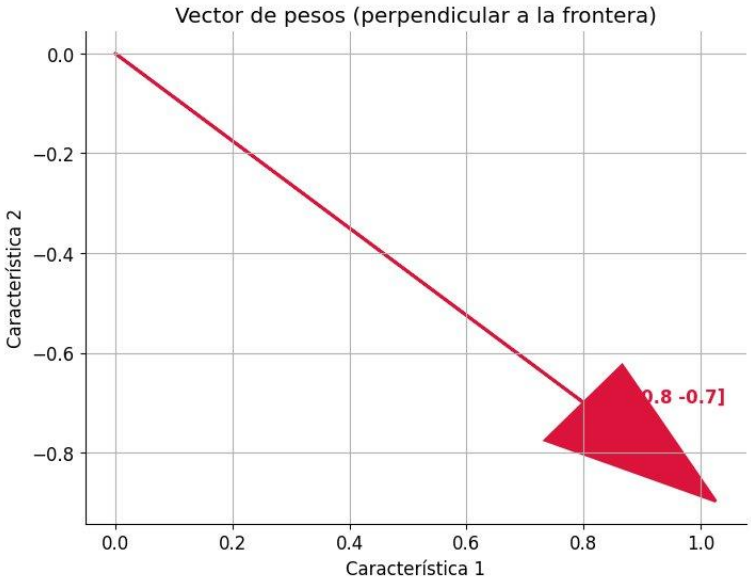

Figura 20. Vector de pesos [0.8, −0.7]: siempre perpendicular a la frontera de decisión. Su dirección indica hacia qué clase apunta el modelo; su magnitud determina la pendiente de la transición.

5.3 Cálculo Manual Paso a Paso

Red neuronal con 3 entradas, capa oculta de 3 neuronas (Sigmoide) y capa de salida de 3 neuronas (Softmax). Vector de entrada normalizado $X = [0.5, 0.3, 0.2]^T$.

Paso 1 — Puntaje Capa Oculta: $Z^{(1)} = W^{(1)}X + b^{(1)}$

Neurona	Cálculo	$Z^{(1)}$

$z_1^{(1)}$	0.2(0.5) + 0.5(0.3) + (−0.1)(0.2) + 0.1	= 0.10 + 0.15 − 0.02 + 0.10 = 0.33
$z_2^{(1)}$	0.8(0.5) + (−0.3)(0.3) + 0.4(0.2) + (−0.2)	= 0.40 − 0.09 + 0.08 − 0.20 = 0.19
$z_3^{(1)}$	(−0.5)(0.5) + 0.1(0.3) + 0.2(0.2) + 0.0	= −0.25 + 0.03 + 0.04 + 0.00 = −0.18

$$\mathbf{Z^{(1)} = [0.33,\ 0.19,\ -0.18]^T}$$

Paso 2 — Activación Sigmoide: $A^{(1)} = \sigma(Z^{(1)})$

Neurona	z	$\sigma(z) = 1/(1+e^{-z})$	$A^{(1)}$
$a_1^{(1)}$	0.33	1/(1+0.7189)	≈ 0.5818
$a_2^{(1)}$	0.19	1/(1+0.8270)	≈ 0.5474
$a_3^{(1)}$	−0.18	1/(1+1.1972)	≈ 0.4551

$$\mathbf{A^{(1)} = [0.5818,\ 0.5474,\ 0.4551]^T}$$

Paso 3 — Puntaje Capa de Salida: $Z^{(2)} = W^{(2)}A^{(1)} + b^{(2)}$

Neurona	Cálculo	$Z^{(2)}$
$z_1^{(2)}$	0.5(0.5818)+(−0.4)(0.5474)+0.1(0.4551)−0.1	= 0.2909−0.2190+0.0455−0.1 = 0.0174
$z_2^{(2)}$	0.2(0.5818)+0.8(0.5474)+(−0.5)(0.4551)+0.2	= 0.1164+0.4379−0.2276+0.2 = 0.5267
$z_3^{(2)}$	(−0.1)(0.5818)+0.3(0.5474)+0.7(0.4551)+0.1	= −0.0582+0.1642+0.3186+0.1 = 0.5246

$$\mathbf{Z^{(2)} = [0.0174,\ 0.5267,\ 0.5246]^T}$$

Paso 4 — Softmax: $A^{(2)}$

Exponenciales: $e^{0.0174} \approx 1.0176$ | $e^{0.5267} \approx 1.6930$ | $e^{0.5246} \approx 1.6895$. Suma = 4.4001.

Clase	Cálculo Softmax	Probabilidad	Interpretación
C1 — Bajo Impacto	1.0176 / 4.4001	P1 ≈ 23.13%	Descartada
C2 — Impacto Medio	1.6930 / 4.4001	P2 ≈ 38.48%	Clase predicha (máxima)
C3 — Impacto Alto	1.6895 / 4.4001	P3 ≈ 38.40%	Muy cercana a C2 (alta incertidumbre)

Resultado: La red clasifica la campaña como Impacto Medio (C2) con P = 38.48%. La proximidad entre C2 y C3 (diferencia de 0.08%) indica alta incertidumbre y sugiere que la campaña podría requerir una segunda evaluación con más datos.

5.4 Preguntas de Análisis

¿Por qué Softmax en la capa de salida?

Softmax garantiza que las probabilidades de todas las clases sumen exactamente 1 ($\Sigma P_k = 1$), haciendo la salida interpretable como distribución de probabilidad real. Una sigmoide independiente no garantiza esta propiedad. Además, Softmax es diferenciable en todos los puntos, permitiendo el backpropagation eficiente durante el entrenamiento.

Rol del Bias: ¿qué sucede si b = 0?

Sin bias, la frontera de decisión de cada neurona queda forzada a pasar por el origen (Figura 18), eliminando el grado de libertad de traslación. El bias cumple la misma función que el intercepto β_0 en regresión lineal: permite al modelo adaptarse a datos cuya separación natural no pasa por el origen. Con bias (Figura 19), la frontera se desplaza libremente y clasifica correctamente todos los puntos del dataset.

6. Fase 5: Evaluación de Modelos — Mantenimiento Predictivo

6.1 Escenarios de Ajuste: Subajuste, Balanceado y Sobreajuste

Se entrenaron tres modelos con diferentes estrategias para ilustrar el espectro de ajuste en clasificación binaria aplicada al control de calidad industrial (clasificar piezas como Apta/Defectuosa):

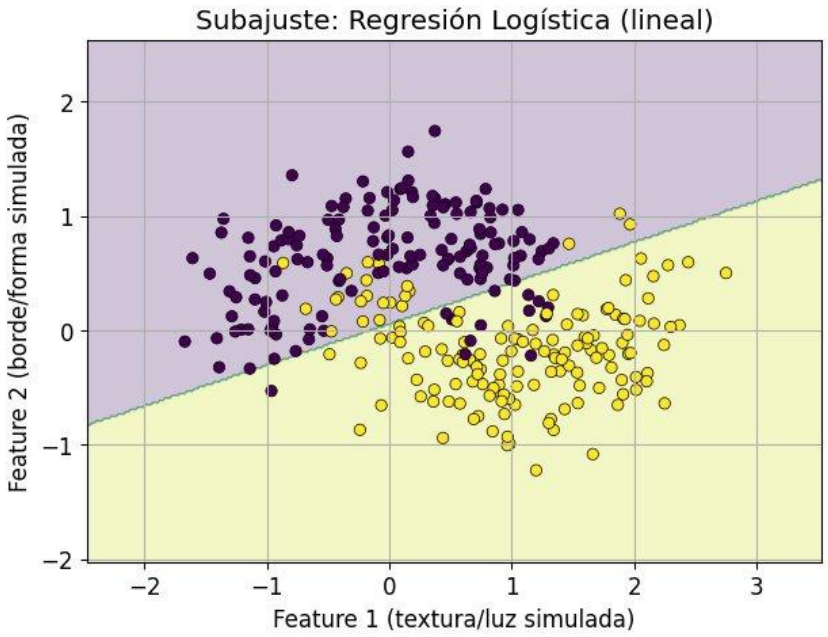

Figura 21. Subajuste — Regresión Logística (frontera lineal): la frontera recta no puede separar adecuadamente las dos clases que presentan distribución no lineal. Alto error en ambos conjuntos.

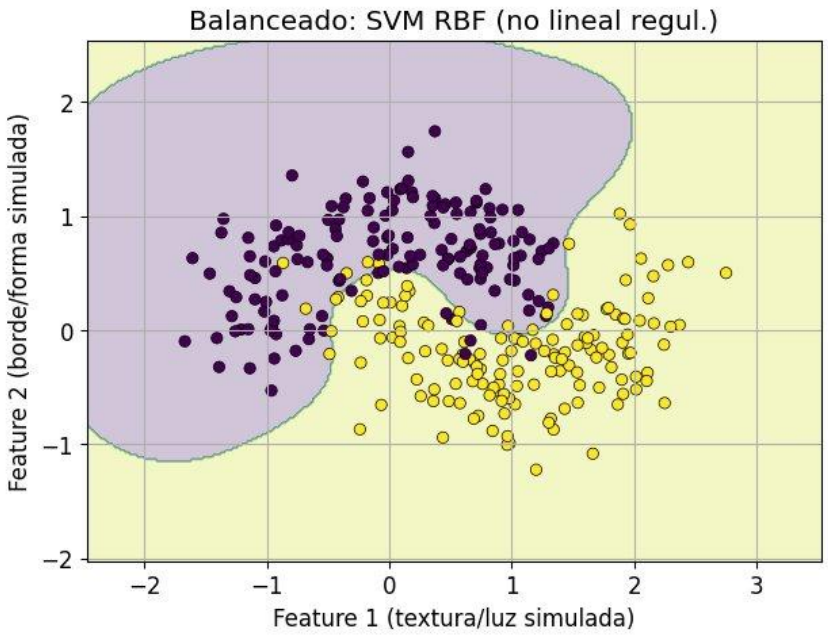

Figura 22. Balanceado — SVM con kernel RBF: la frontera curva se adapta a la distribución real de los datos sin memorizar el ruido. Mejor equilibrio sesgo-varianza de los tres modelos.

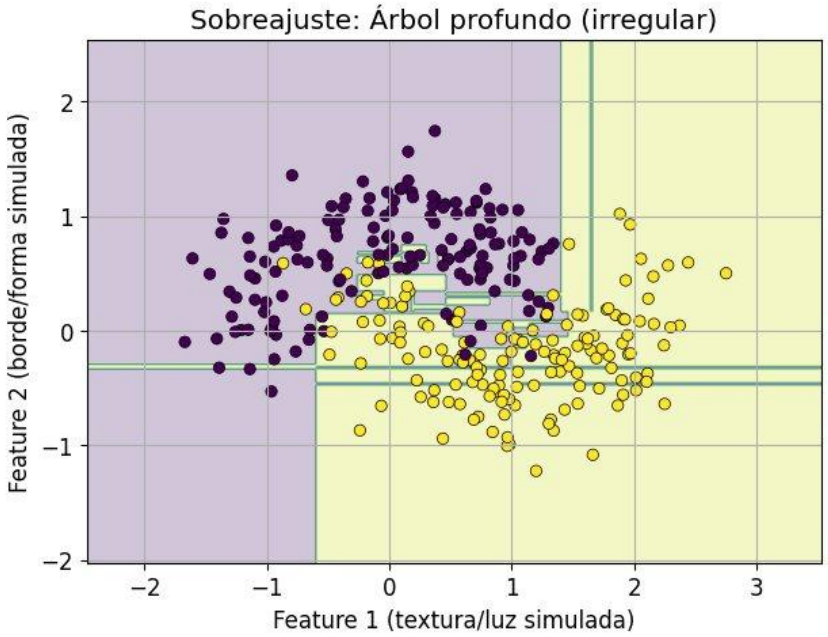

Figura 23. Sobreajuste — Árbol de Decisión profundo sin poda: la frontera irregular memoriza el ruido de entrenamiento (error de train ≈ 0%) pero generaliza pobremente en test. Típico de modelos sin regularización.

6.2 Curvas de Aprendizaje

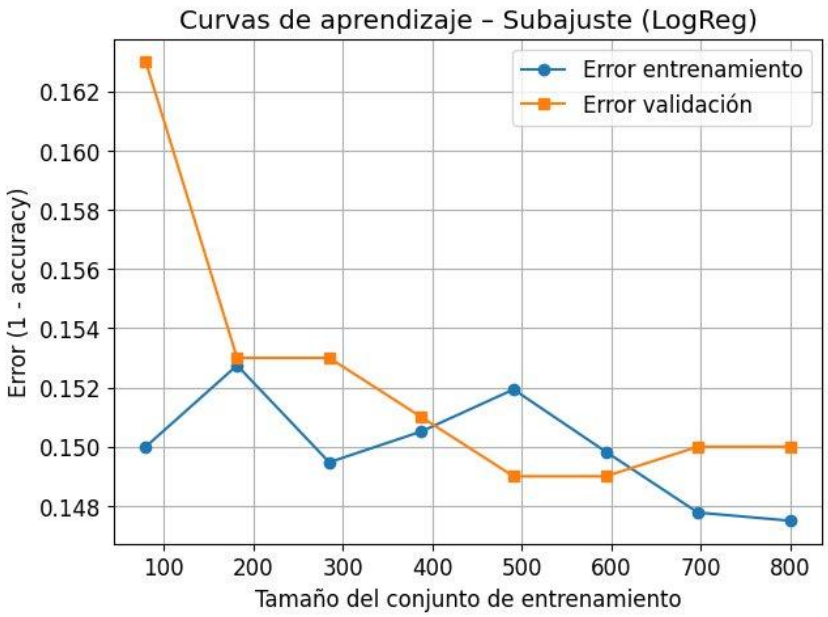

Figura 24. Curvas de aprendizaje — Subajuste (LogReg): error de entrenamiento y validación convergen ambos alrededor del 15%, sin brecha significativa entre ellos. Señal característica de subajuste: el modelo es demasiado simple para capturar la complejidad del problema.

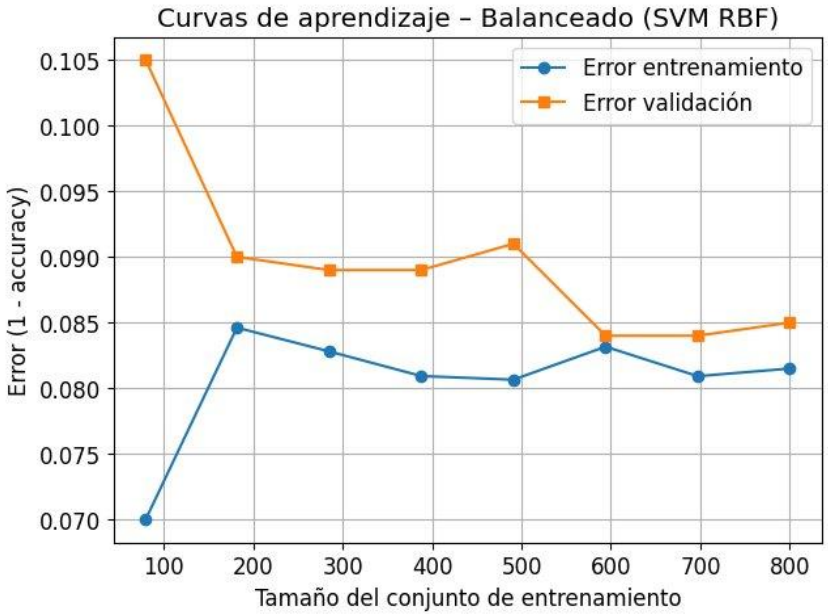

Figura 25. Curvas de aprendizaje — Balanceado (SVM RBF): error de entrenamiento ≈ 8% y validación ≈ 9%, con brecha pequeña y estable. El modelo generaliza bien. A partir de ≈600 muestras las curvas convergen, indicando que no se necesita más datos para mejorar.

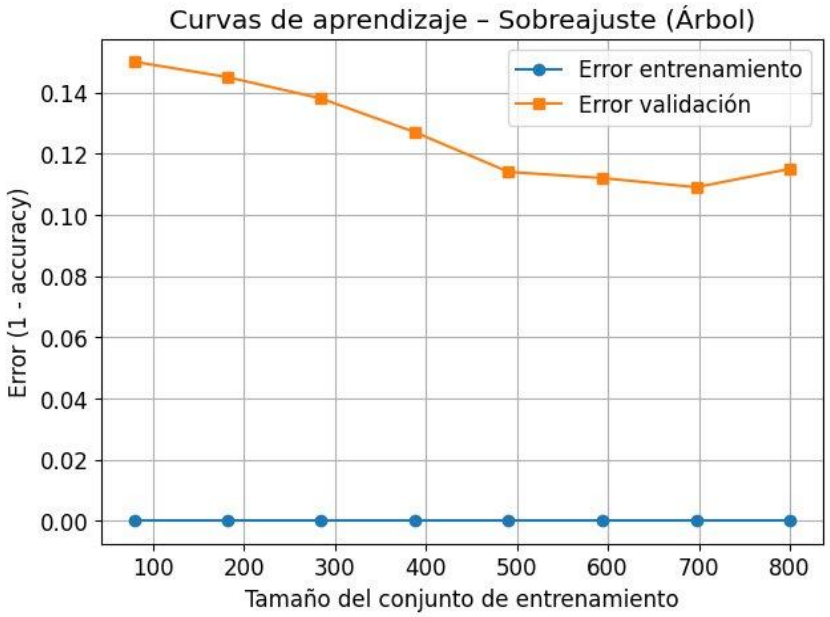

Figura 26. Curvas de aprendizaje — Sobreajuste (Árbol profundo): error de entrenamiento = 0% (memorización perfecta) mientras el error de validación oscila entre 10-15%. La brecha amplia y persistente es la firma inequívoca del sobreajuste. Solución: podar el árbol o limitar la profundidad máxima.

6.3 Efecto de la Regularización

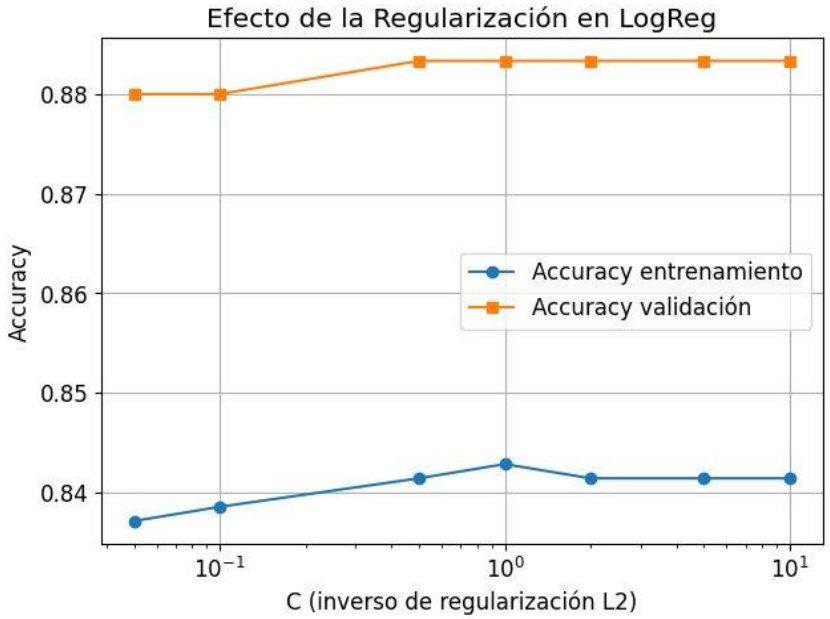

Figura 27. Efecto de la regularización en Regresión Logística (C = 1/λ). A medida que C aumenta (menor regularización), el accuracy de validación mejora hasta ≈88.3% y luego se estabiliza. El accuracy de entrenamiento permanece bajo (≈84%), confirmando que el problema de subajuste de LogReg no se resuelve solo con regularización — se necesita un modelo más flexible (SVM RBF o MLP).

6.4 Matrices de Confusión Comparativas

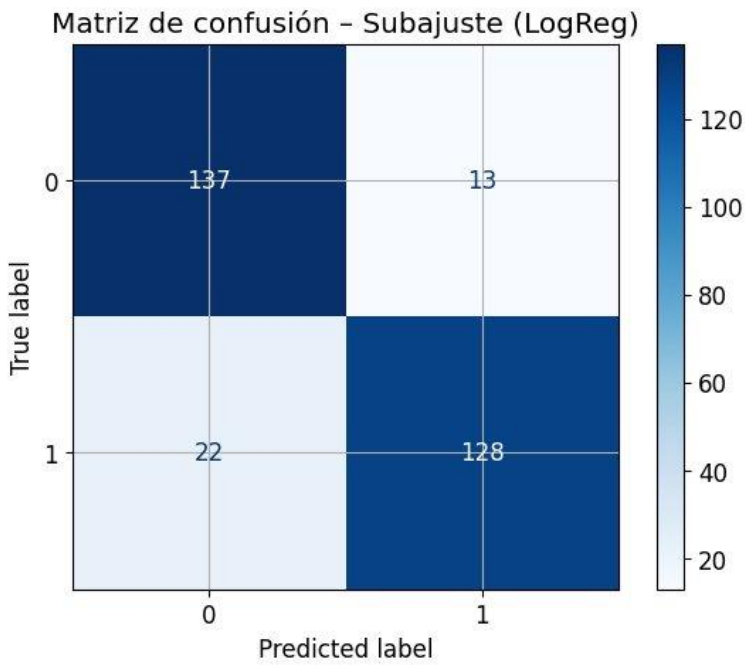

Figura 28. CM Subajuste (LogReg): TN=137, FP=13, FN=22, TP=128. Recall = 128/150 = 85.3%. Alta tasa de falsos negativos (22 fallos no detectados).

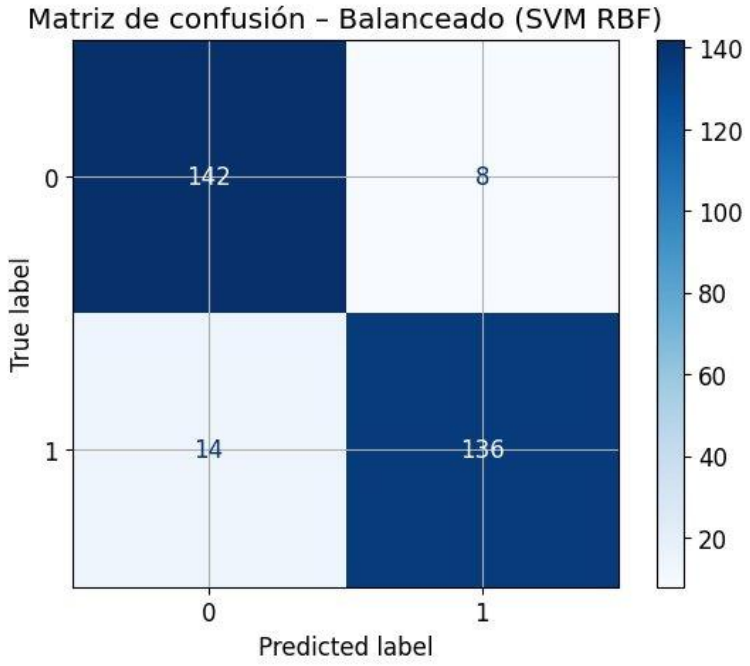

Figura 29. CM Balanceado (SVM RBF): TN=142, FP=8, FN=14, TP=136. Recall = 136/150 = 90.7%. Mejor detección de fallos reales con menor tasa de falsas alarmas.

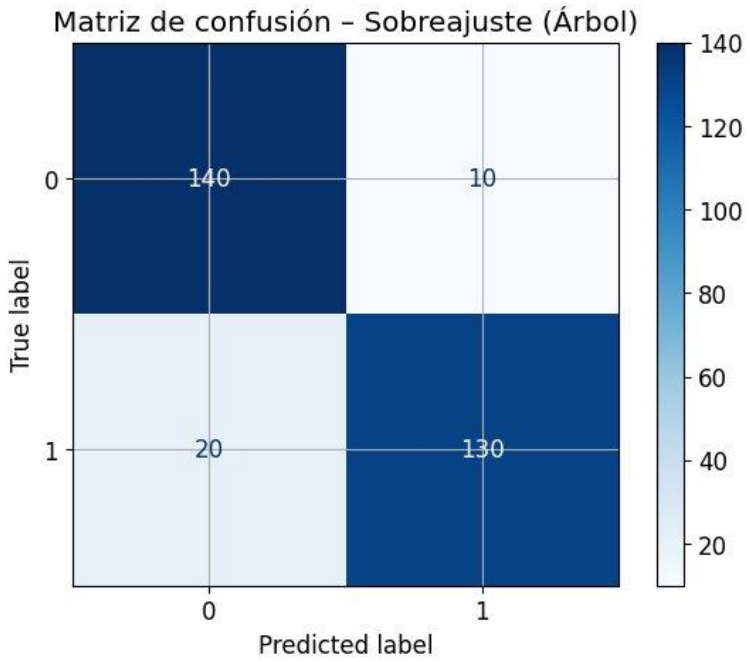

Figura 30. CM Sobreajuste (Árbol profundo): TN=140, FP=10, FN=20, TP=130. Recall = 130/150 = 86.7%. A pesar del sobreajuste aparente en entrenamiento, el rendimiento en test es inferior al SVM RBF.

6.5 Auditoría del Modelo — Caso Mantenimiento Predictivo

El modelo final de mantenimiento predictivo (árbol de decisión optimizado y podado) fue evaluado en el conjunto de prueba. La matriz de confusión de auditoría presenta los siguientes valores:

Figura 31. Matriz de Confusión — Auditoría del Modelo. TN=177 (piezas aptas correctamente identificadas), FP=3 (falsas alarmas), FN=2 (defectos ocultos), TP=18 (defectos correctamente detectados).

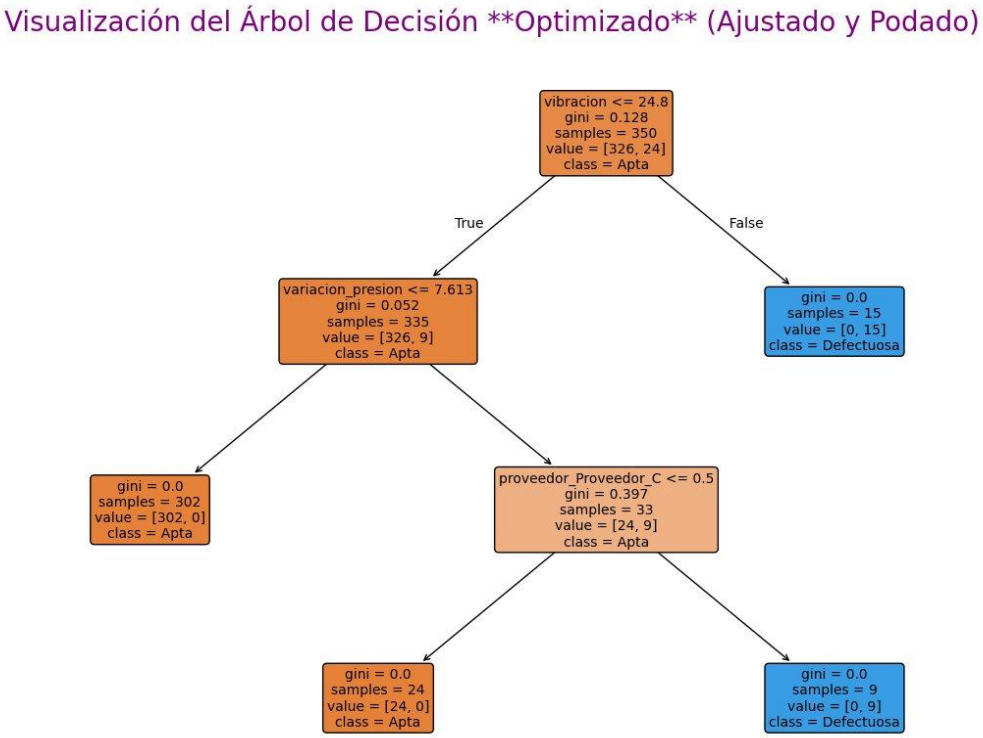

Figura 32. Árbol de Decisión Optimizado y Podado. Regla principal: si vibración ≤ 24.8, rama izquierda (mayormente Apta). Si vibración > 24.8, todos los 15 casos son Defectuosos (Gini = 0, nodo puro). La segunda regla involucra variacion_presion ≤ 7.613 y proveedor_Proveedor_C ≤ 0.5 para casos intermedios.

6.6 Cálculo Manual de Métricas

A partir de los valores de la auditoría: TN = 177, FP = 3, FN = 2, TP = 18.

Métrica	Fórmula	Cálculo	Resultado	Interpretación operativa
Precision	TP/(TP+FP)	18/(18+3)	85.71%	De cada 21 alertas emitidas, 18 corresponden a defectos reales. Solo 3 inspecciones innecesarias por cada 21 alertas.
Recall (Sensibilidad)	TP/(TP+FN)	18/(18+2)	90.00%	El modelo captura 9 de cada 10 defectos reales. Solo 2 defectos ocultos pasan sin detectar.
Accuracy	(TP+TN)/Total	(18+177)/200	97.50%	Alta exactitud global, pero en contexto de desbalance puede ser engañosa.
F1-Score	2·P·R/(P+R)	2·0.857·0.90/1.757	87.80%	Equilibrio armónico entre Precision y Recall. Métrica robusta para clases desbalanceadas.

Costo Total	(FN×5000)+(FP×200)	(2×5000)+(3×200)	USD 10,600	Costo mensual estimado del modelo: $10,000 por 2 defectos no detectados + $600 por 3 falsas alarmas.

6.7 Preguntas de Reflexión y Negocio

Análisis de Pesos de Clase

Al asignar un peso 14.7× mayor a los errores de Clase 1 (Fallo), el algoritmo corrige la tendencia natural del modelo a ignorar los fallos por ser raros. Matemáticamente, cada error de clasificación en un fallo real contribuye 14.7 veces más al gradiente, forzando al modelo a ajustar sus pesos principalmente para reducir los falsos negativos. Desde la perspectiva del negocio manufacturero, este ajuste refleja la asimetría de costos: FN = USD 5,000 vs. FP = USD 200.

La Trampa de la Exactitud (Accuracy Paradox)

Un modelo que prediga siempre 'Apta' obtendría Accuracy ≈ 93% (177/190 casos Apta en el dataset), sin detectar ningún defecto. El Recall del 90% del modelo auditado frente al 0% del modelo trivial demuestra que el Accuracy es una métrica engañosa en contextos de desbalance. Las métricas relevantes en manufactura son Recall, Precision y el costo económico de cada tipo de error.

Trade-off Recall vs. Precision en manufactura crítica

Con los costos calculados (FN = USD 5,000, FP = USD 200), la estrategia óptima prioriza el Recall. Un modelo con Recall del 98% y Precision del 60% generaría: si ocurren 10 defectos/mes y solo 0.2 pasan sin detectar → USD 1,000 en pérdidas + USD 1,600 en falsas alarmas = USD 2,600 total. El modelo actual (Recall 90%, Precision 85.7%) genera USD 10,600 mensuales. Aumentar el umbral de decisión para priorizar Recall reduciría los FN a costa de más FP, pero el análisis de costos favorece claramente esta estrategia.

Árbol de Decisión Optimizado: Reglas de Negocio

La visualización del árbol podado (Figura 32) revela reglas de negocio directamente accionables: (1) Si vibración > 24.8, la pieza es Defectuosa con certeza absoluta (Gini = 0, 15 casos, 100% precisión) — esta regla sola podría implementarse como alarma automática en la línea de producción. (2) Para piezas con vibración ≤ 24.8 y variacion_presion > 7.613, el proveedor determina la clasificación: Proveedor C produce defectos (9 de 9 casos), mientras los demás proveedores producen piezas aptas (24 de 24 casos). Esta segunda regla tiene implicaciones directas de gestión de proveedores.

Curva de Aprendizaje y Épocas Suficientes

En el caso del árbol optimizado, la convergencia se logra con el dataset completo de entrenamiento. El parámetro ccp_alpha de poda determina la complejidad final del árbol, análogo al λ de Ridge. Un árbol sin poda (Figura 26) memoriza perfectamente el entrenamiento (error = 0%) pero generaliza pobremente. El árbol optimizado (Figura 32) con solo 4 niveles y 5 nodos hoja alcanza Recall = 90% y Precision = 85.7%, demostrando que la simplicidad estructural es preferible a la complejidad cuando los datos son limitados.

7. Conclusiones

Este taller ha recorrido el ciclo completo de un proyecto de analítica avanzada aplicada a la ingeniería industrial, desde la fundamentación matemática hasta la toma de decisiones de negocio:

- Fase 1 — La Regresión Ridge no es una modificación empírica de OLS, sino una solución matemáticamente rigurosa que garantiza la invertibilidad del sistema mediante la adición de λI y controla el equilibrio sesgo-varianza a través de λ. Los heatmaps de RMSE CV confirman que alpha ≈ 1 es la zona óptima de regularización para el dataset de bicicletas, y la visualización 3D evidencia la diferencia cualitativa entre las restricciones L1 (Lasso, induce sparsidad) y L2 (Ridge, contracción uniforme).
- Fase 2 — El descenso de gradiente es extremadamente sensible a η y al punto de inicio. En la función de Himmelblau, la multimodalidad demuestra empíricamente por qué en problemas industriales de alta dimensionalidad es indispensable la estrategia multi-start y los optimizadores adaptativos como Adam.
- Fase 3 — El MLP optimizado supera significativamente al baseline OLS al capturar relaciones no lineales entre hora del día y demanda de bicicletas. La regularización alpha y el early stopping son los mecanismos más efectivos para prevenir el sobreajuste. El MAE estimado de 40-60 bicicletas/hora define el buffer de seguridad para decisiones de redistribución de flota.
- Fase 4 — La propagación hacia adelante manual demuestra que una red neuronal es una composición de transformaciones lineales y no lineales. El bias y la función Softmax son componentes críticos: sin bias la frontera queda forzada al origen (Figura 18); sin Softmax las probabilidades no suman 1 y no son comparables entre clases.
- Fase 5 — El modelo de mantenimiento predictivo auditado (Recall = 90%, Precision = 85.7%, Costo mensual = USD 10,600) demuestra que las métricas orientadas al negocio superan en relevancia a las métricas genéricas como el Accuracy. Las reglas del árbol podado (vibración > 24.8 = Defectuosa, Proveedor C = factor de riesgo) son directamente accionables en la operación de planta, ilustrando el valor de la interpretabilidad en sistemas de decisión industrial.

8. Referencias

Fanaee-T, H., & Gama, J. (2013). Event labeling combining ensemble detectors and background knowledge. Progress in Artificial Intelligence, 2(2-3), 113–127. https://doi.org/10.1007/s13748-013-0040-3

Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. Technometrics, 12(1), 55–67.

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. Journal of the Royal Statistical Society: Series B, 58(1), 267–288.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://scikit-learn.org

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. https://www.deeplearningbook.org

Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.

Ruder, S. (2016). An overview of gradient descent optimization algorithms. arXiv:1609.04747.